



Laboratório 4: Outros conceitos em Kotlin

6 de agosto de 2026

Este é o quarto laboratório da disciplina de Programação para Dispositivos Móveis. O objetivo desta atividade é praticar outros conceitos da linguagem Kotlin.

Exercícios

1. Crie um diretório chamado `lab_n`, onde `n` é o número do laboratório, e dentro dele crie um projeto Kotlin.
2. Para cada questão, crie um arquivo Kotlin separado, nomeando-o de acordo com a questão (por exemplo, `Exercicio1.kt`, `Exercicio2.kt`, etc.).
3. Implemente os exercícios abaixo.

Sistema de pedidos (delivery)

Implemente um pequeno sistema em modo console para modelar pedidos de uma lanchonete/delivery.

1. Crie a enumeração `CategoriaItem` com, no mínimo, os valores: `PIZZA`, `BEBIDA`, `SOBREMESA`. Cada valor deve ter um atributo `descricao: String` e uma função `descricao()` que retorne uma string amigável.
2. Crie a enumeração `StatusPedido` com, no mínimo: `NOVO`, `EM_PREPARO`, `SAIU_PARA_ENTREGA`, `ENTREGUE`, `CANCELADO`. Inclua uma função que ajude no controle de fluxo, por exemplo: `podeCancelar(): Boolean` OU `podeAvancar(): Boolean`.
3. Crie uma *data class* `ItemPedido` com: `nome: String`, `categoria: CategoriaItem`, `preco: Double`.
4. Crie uma *data class* `Pedido` com: `id: Int`, `cliente: String`, `itens: List<Pair<ItemPedido, Int>>` (par (*item*, *quantidade*)), e `status: StatusPedido` (com valor inicial `NOVO`).
5. Faça, no `main()`, uma coleção de pedidos com pelo menos 3 pedidos e itens variados. Ao imprimir o pedido, use desestruturação em pares, por exemplo: `for ((item, qtd) in pedido.itens) {...}`.
6. Crie pelo menos duas extensões (função e/ou propriedade) para facilitar o código, por exemplo:
 - `fun Double.emReais(): String` (formate como moeda ou com 2 casas decimais);
 - `fun List<Pair<ItemPedido, Int>>.totalItens(): Int`;
 - `fun List<Pair<ItemPedido, Int>>.subtotal(): Double`;
 - `val Pedido.resumo: String` (resumo com id, cliente, status e subtotal).
7. Para praticar *Triple*, crie uma função: `fun resumoPagamento(subtotal: Double, taxaEntrega: Double): Triple<Double, Double, Double>` que retorne (subtotal, taxaEntrega, total). Use desestruturação ao consumir o resultado.

Repositório genérico e tratamento de exceções

Agora você vai armazenar e consultar pedidos de forma segura.

1. Implemente uma classe genérica `Repositorio<T>` baseada em `MutableMap<Int, T>` com operações mínimas: `adicionar(id: Int, valor: T)`, `obter(id: Int): T`, `remover(id: Int)`, `listar(): List<T>`.
2. Valide pré-condições com `require`, por exemplo: `id > 0`, e mensagens claras em caso de erro.
3. Crie uma exceção personalizada `NaoEncontradoException` (ou equivalente) e lance-a quando `obter(id)` falhar.
4. No `main()`, demonstre o uso de `try/catch/finally`:
 - tentando obter um pedido inexistente;
 - tentando adicionar um pedido com `id` inválido;
 - garantindo que o programa não encerre abruptamente.

Processamento assíncrono de pedidos

Simule o processamento de pedidos (como se fosse acesso a rede/serviço), comparando execução sequencial e concorrente.

1. Crie um *singleton* com object `Configuracao` contendo, por exemplo: `taxaEntrega: Double`, `tempoBaseMs: Long` e uma função `exibirConfiguracao()`.
2. Em `Pedido`, crie uma propriedade calculada com delegação lazy: `val total: Double by lazy { ... }` que use o `subtotal + Configuracao.taxaEntrega`. (Dica: mantenha a lista de itens imutável.)
3. Crie uma *sealed class* `ResultadoProcessamento` com, no mínimo: `Sucesso(...)` e `Falha(...)` para representar o resultado de processar um pedido.
4. Crie uma função *suspend* `processarPedido(pedido: Pedido): ResultadoProcessamento` que:
 - use `require(pedido.status != CANCELADO)` e trate o erro;
 - use `delay(...)` para simular tempo de processamento (por exemplo, proporcional à quantidade total de itens);
 - avance o status do pedido quando fizer sentido (ex.: `NOVO -> EM_PREPARO -> ...`).
5. Adicione no projeto a dependência de coroutines:

```
dependencies {  
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.9.0")  
}
```

6. No `main()`, usando `runBlocking`:
 - Processe uma lista de pedidos sequencialmente (um por vez) e meça o tempo total.
 - Processe os mesmos pedidos concorrentemente usando `coroutineScope + async` e `await` e meça o tempo total.
 - Compare os tempos e imprima um relatório final com: total de pedidos, quantos sucesso/falha, e o tempo em cada modo.



Entrega

- Data: 13/8/2026.